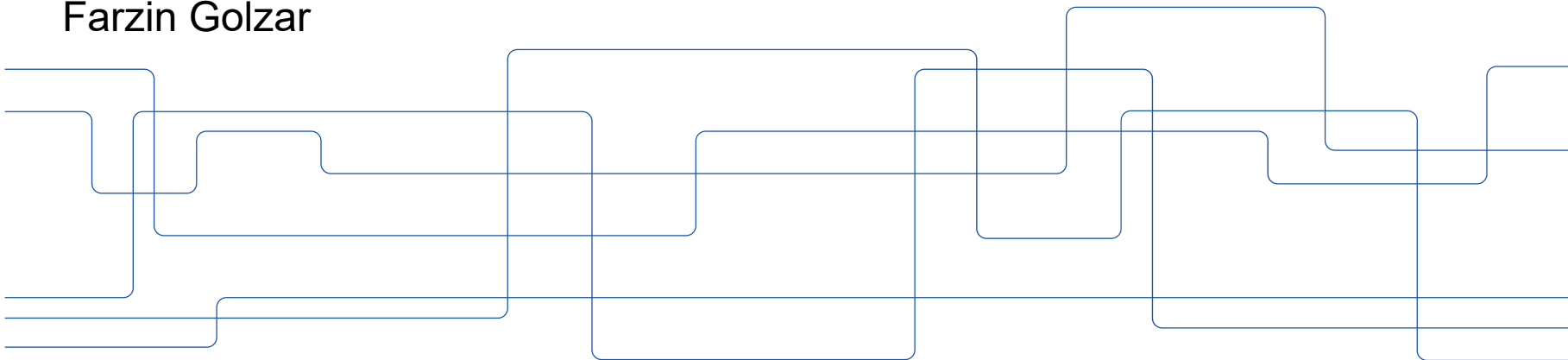


Deep Neural Networks (Build a deep L-layer neural network)

Farzin Golzar

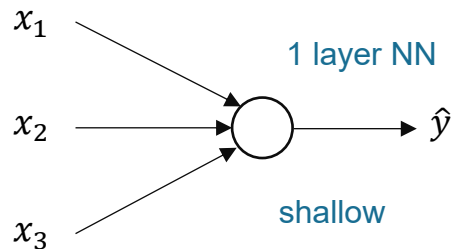




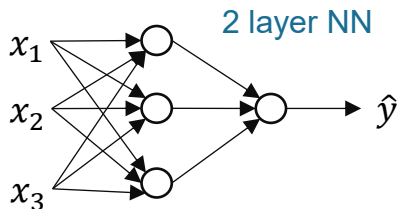
Learning Objectives

- What is a deep neural network?
- Describe the successive block structure of a deep neural network
- Build a deep L-layer neural network
- Analyze matrix and vector dimensions to check neural network implementations
- Use a cache to pass information from forward to back propagation
- Explain the role of hyperparameters in deep learning

What is a deep neural network?

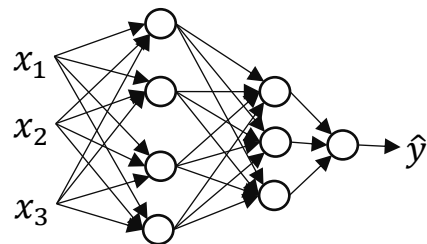


Logistic Regression

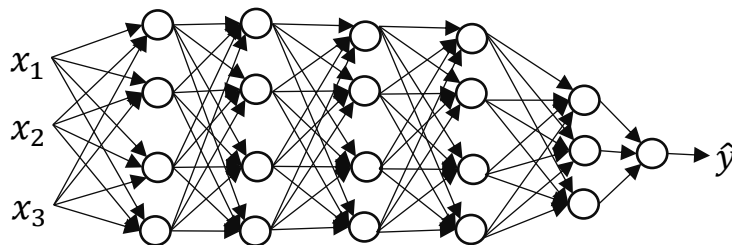


1 hidden layer

It would be reasonable to try logistic regression, try one and then two hidden layers, and view the number of hidden layers



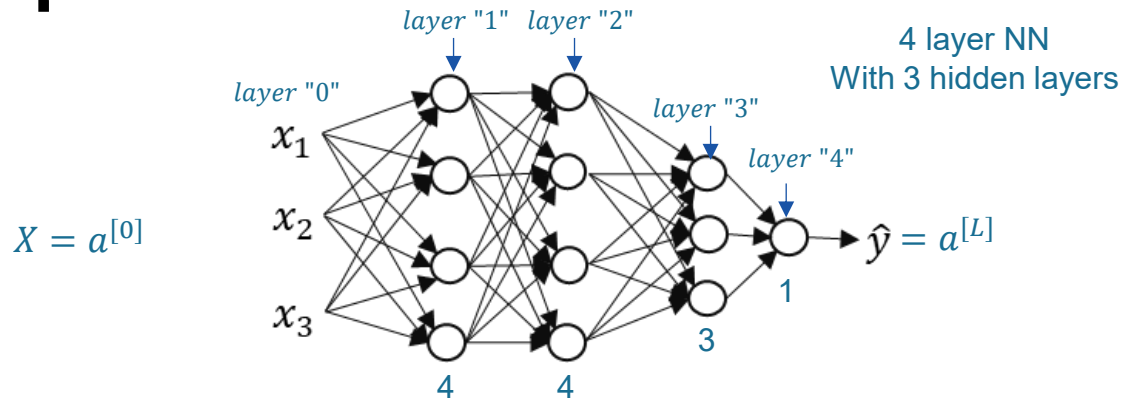
2 hidden layers



5 hidden layers

deep

Deep neural network notation



$L = 4$ (#layers)

$n^{[l]} = \text{\#units in layer } l$

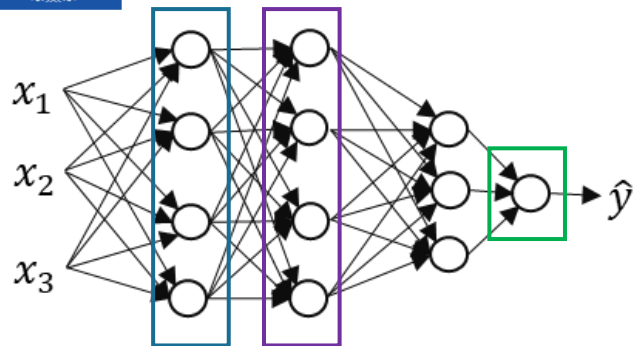
$n^{[1]} = 4, n^{[2]} = 4, n^{[3]} = 3, n^{[4]} = n^{[L]} = 1$

$n^{[0]} = n_x = 3$

$a^{[l]} = \text{activations in layer } l$

$a^{[l]} = g^{[l]}(z^{[l]})$ $w^{[l]} = \text{weights for } Z^{[l]}$ $b^{[l]}$

Forward propagation in a deep network



$$\begin{aligned}
 x: z^{[1]} &= w^{[1]}x + b^{[1]} \\
 a^{[1]} &= g^{[1]}(z^{[1]}) \\
 z^{[2]} &= w^{[2]}a^{[1]} + b^{[2]} \\
 a^{[2]} &= g^{[2]}(z^{[2]}) \\
 &\dots \\
 z^{[4]} &= w^{[4]}a^{[3]} + b^{[4]} \\
 a^{[4]} &= g^{[4]}(z^{[4]}) = \hat{y}
 \end{aligned}$$

General rule for 1 training example:

$$z^{[l]} = w^{[l]}a^{[l-1]} + b^{[l]}$$

$$a^{[l]} = g^{[l]}(z^{[l]})$$

Vectorization:

$$Z^{[1]} = W^{[1]}X + b^{[1]}$$

$$A^{[1]} = g^{[1]}(Z^{[1]})$$

$$Z^{[2]} = W^{[2]}A^{[1]} + b^{[2]}$$

$$A^{[2]} = g^{[2]}(Z^{[2]})$$

$$A^{[4]} = g^{[4]}(Z^{[4]}) = \hat{Y}$$

General rule for all training example:

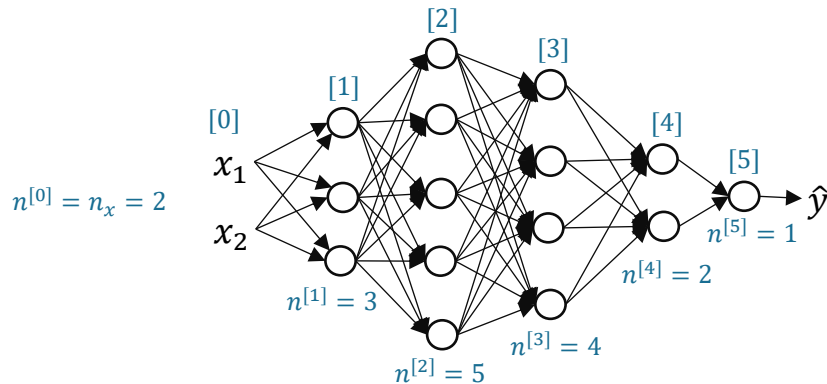
$$Z^{[l]} = W^{[l]}A^{[l-1]} + b^{[l]}$$

$$A^{[l]} = g^{[l]}(Z^{[l]})$$

for $l = 1 - 4$

It is always good to get rid of for loops. However, there is no efficient way to get rid of this one.

Matrix dimensions



$$z^{[1]} = w^{[1]} \cdot x + b^{[1]}$$

$(3,1) \quad (3,2) \quad (2,1) \quad (3,1)$
 $(n^{[1]}, 1) \quad (n^{[0]}, 1)$
 $(n^{[1]}, n^{[0]})$

$$\begin{bmatrix} z \\ z \\ z \end{bmatrix} = \begin{bmatrix} w & w \\ w & w \\ w & w \end{bmatrix} \cdot \begin{bmatrix} x \\ x \end{bmatrix}$$

$$w^{[1]}: (n^{[1]}, n^{[0]})$$

$$w^{[2]}: (n^{[2]}, n^{[1]}) = (5, 3)$$

$$z^{[2]} = w^{[2]} \cdot a^{[1]} + b^{[2]}$$

$(5,1) \quad (5,3) \quad (3,1) \quad (5,1)$

$$w^{[3]}: (4, 5)$$

$$w^{[4]}: (2, 4)$$

$$w^{[5]}: (1, 2)$$

$$w^{[l]}: (n^{[l]}, n^{[l-1]})$$

$$b^{[l]}: (n^{[l]}, 1)$$

$$dw^{[l]}: (n^{[l]}, n^{[l-1]})$$

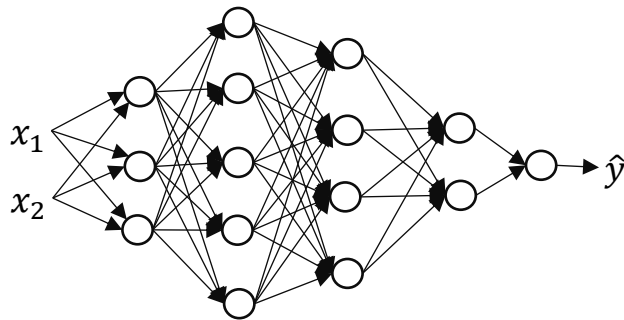
$$db^{[l]}: (n^{[l]}, 1)$$



Note:

The dimension of these parameters would be **similar** both for single or multiple training examples

Vectorized implementation



$$\underset{(n^{[1]}, 1)}{z^{[1]}} = \underset{(n^{[1]}, n^{[0]})}{w^{[1]}} \cdot \underset{(n^{[0]}, 1)}{x} + \underset{(n^{[1]}, 1)}{b^{[1]}}$$

Vectorized

$$\underset{(n^{[1]}, m)}{Z^{[1]}} = \underset{(n^{[1]}, n^{[0]})}{W^{[1]}} \cdot \underset{(n^{[0]}, m)}{X} + \underset{(n^{[1]}, 1)}{b^{[1]}}$$

broadcasting

$$z^{[l]}, a^{[l]}: (n^{[l]}, 1)$$

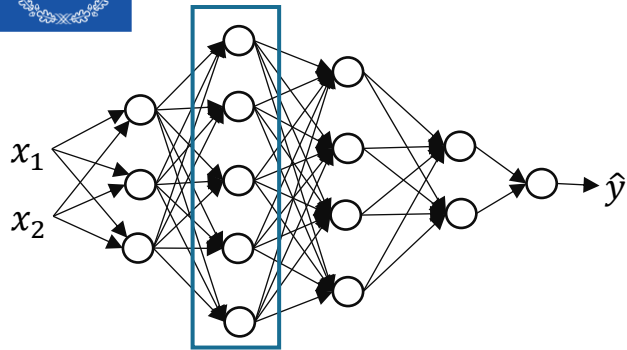
Vectorized

$$Z^{[l]}, A^{[l]}: (n^{[l]}, m)$$

$$dZ^{[l]}, dA^{[l]}: (n^{[l]}, m)$$

As you work through your code and make sure that all the **matrices or dimensions are consistent**, that will usually help you go some ways towards eliminating some class of possible bugs.

Blocks of deep learning



Layer l : $w^{[l]}, b^{[l]}$

Forward: Input $a^{[l-1]}$, output $a^{[l]}$

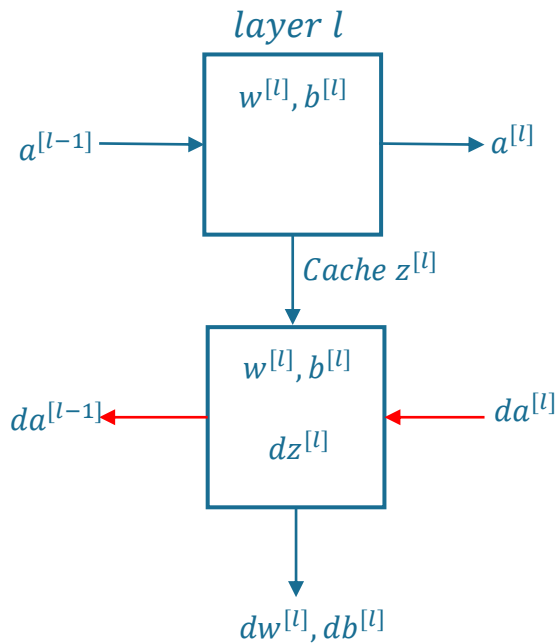
$$z^{[l]} = w^{[l]}a^{[l-1]} + b^{[l]} \quad \text{cache } z^{[l]}$$

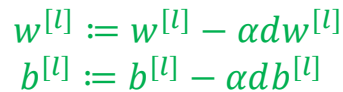
$$a^{[l]} = g^{[l]}(z^{[l]})$$

Backward: Input $da^{[l]}$, output $da^{[l-1]}$

cache $z^{[l]}$

$dw^{[l]}$
 $db^{[l]}$







Forward propagation for layer l

Input $a^{[l-1]}$

Output $a^{[l]}$, cache $(z^{[l]})$

$w^{[l]}, b^{[l]}$

$$z^{[l]} = w^{[l]} \cdot a^{[l-1]} + b^{[l]}$$

$$a^{[l]} = g^{[l]}(z^{[l]})$$

Vectorized:

$$Z^{[l]} = W^{[l]} \cdot A^{[l-1]} + b^{[l]}$$

$$A^{[l]} = g^{[l]}(Z^{[l]})$$



Backward propagation for layer l

Input $da^{[l]}$

Output $da^{[l-1]}, dW^{[l]}, db^{[l]}$

$$dz^{[l]} = da^{[l]} * g^{[l]}(z^{[l]})$$

$$dw^{[l]} = dz^{[l]} \cdot a^{[l-1]T}$$

$$db^{[l]} = dz^{[l]}$$

$$da^{[l-1]} = w^{[l]T} dz^{[l]}$$

Vectorized:

$$dZ^{[l]} = dA^{[l]} * g^{[l]}(Z^{[l]})$$

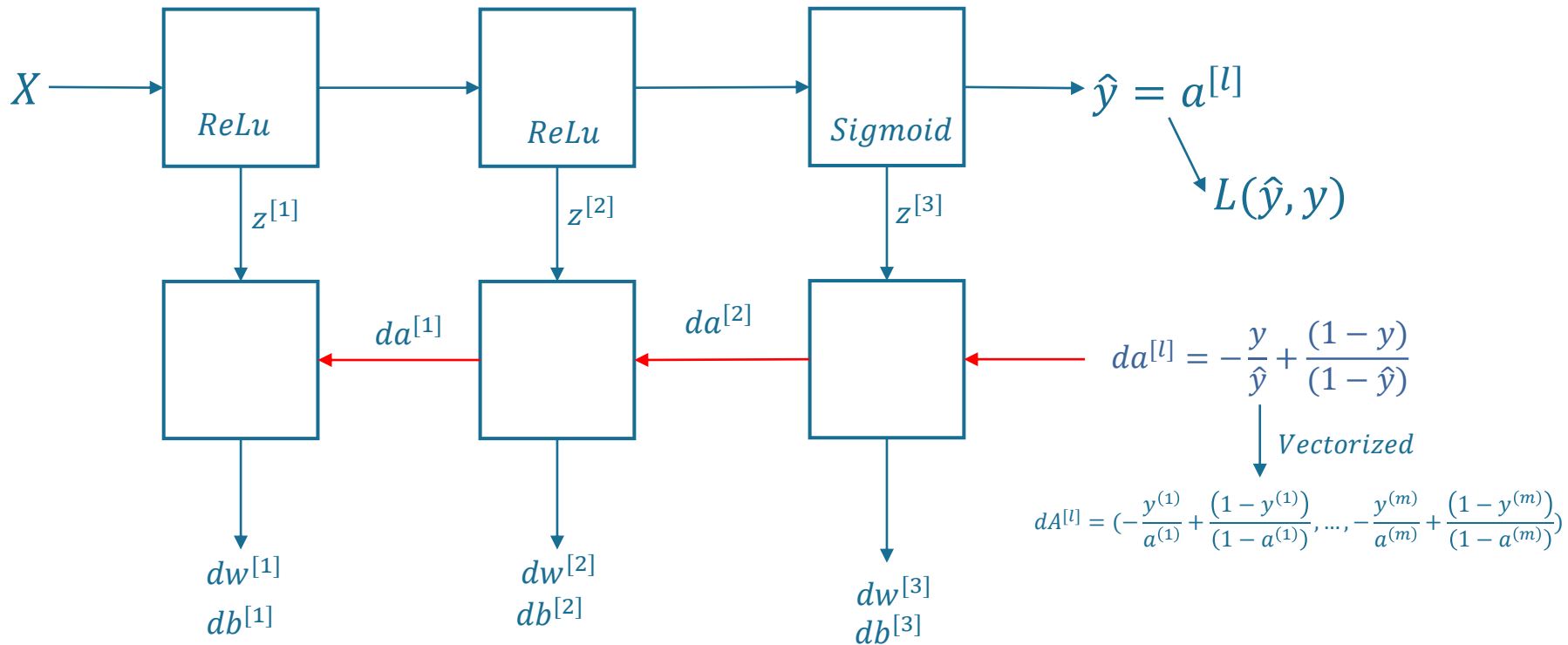
$$dW^{[l]} = \frac{1}{m} dZ^{[l]} \cdot A^{[l-1]T}$$

$$db^{[l]} = \frac{1}{m} np.sum(dZ^{[l]}, axis = 1, keepdims = True)$$

$$dA^{[l-1]} = W^{[l]T} dZ^{[l]}$$

Summary

When you've done the **programming exercise**, it will feel a bit more concrete to you.

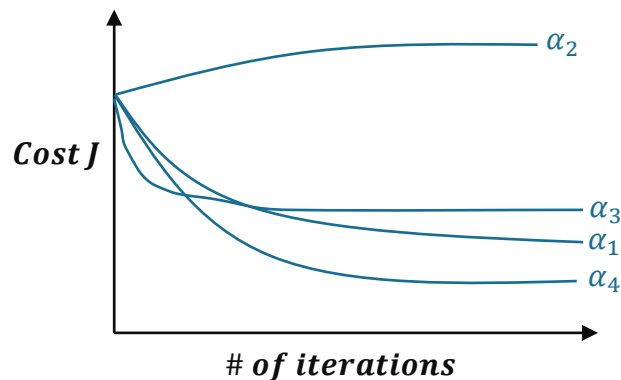
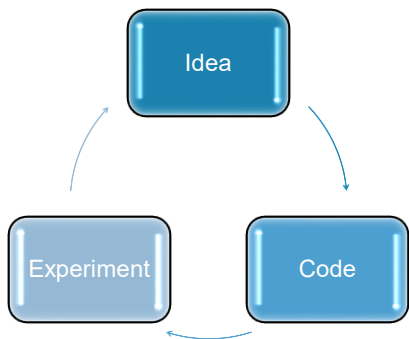


Parameters VS hyperparameters

Parameters: $W^{[1]}, b^{[1]}, W^{[2]}, b^{[2]}, W^{[3]}, b^{[3]} \dots$

Hyperparameters: $\left\{ \begin{array}{l} \text{Learning rate, } \alpha \\ \text{\#iterations} \\ \text{\#hidden layer } l \\ \text{\#hidden units, } n^{[1]}, n^{[2]}, \dots \\ \text{Choice of activation function} \end{array} \right.$

Hyperparameters control parameters





Hyperparameter optimization (HPO)

Method	Basic idea
Manual Search	Try values based on experience
Grid Search	Try every predefined combination
Random Search	Randomly sample combinations
Bayesian Optimization	Use results from previous trials to intelligently choose the next trial
Evolutionary Algorithms	Evolve good combinations using ideas inspired by natural selection
Hyperband / Successive Halving	Stop poor configurations early and spend more resources on promising ones



Grid search

An easy example

Suppose we define:

Learning rate = [0.1, 0.01, 0.001]

Neurons = [32, 64, 128]



Grid search tries:

0.1 + 32
0.1 + 64
0.1 + 128

0.01 + 32
0.01 + 64
0.01 + 128

0.001 + 32
0.001 + 64
0.001 + 128



That's 9 neural networks to train.

We evaluate them on the validation set and select the configuration with the best validation performance.



Random Search

Instead of testing everything, randomly select combinations:

Trial 1 $\rightarrow$ LR = 0.0032, neurons = 64

Trial 2 $\rightarrow$ LR = 0.0007, neurons = 128

Trial 3 $\rightarrow$ LR = 0.018, neurons = 32

...

This can be surprisingly effective, especially when only a few hyperparameters strongly affect performance.



Bayesian Optimization

This is a more intelligent search.

Imagine:

Trial 1: learning rate 0.1 $\rightarrow$ poor performance

Trial 2: learning rate 0.001 $\rightarrow$ good performance

Trial 3: learning rate 0.003 $\rightarrow$ very good performance

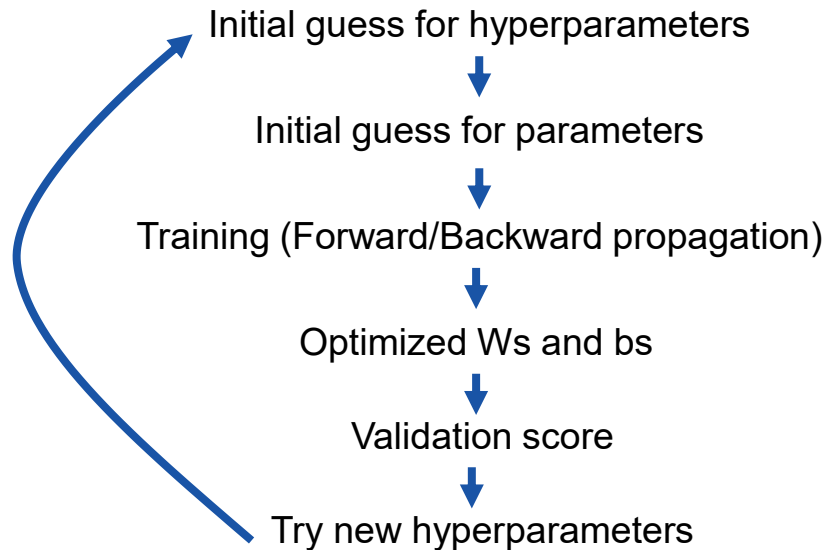
The algorithm uses those previous results to decide:

"It looks like the promising region is around 0.001–0.003. Let's explore there more."

So unlike random/grid search, **each experiment informs the next experiment.**

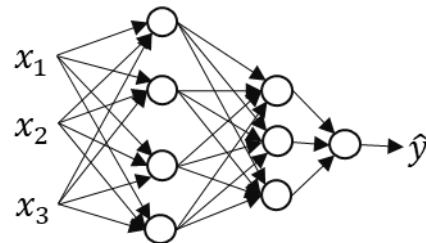


General Process



What to do next

- Go to CANVAS
- Solve HW3_1 where you build all necessary functions
- Solve HW3_2 where you put together all functions





References

- [Speech and Language Processing](#). Daniel Jurafsky & James H. Martin. Draft of January 7, 2023.
- [Deeplearning.ai](#)
- [Implementing-a-neural-network-from-scratch/](#)